BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Họ và tên	**NGUYỄN LƯƠNG HỒNG BÚP**
MSSV / Lớp	**3124720009**

Mục tiêu học tập

- Mô hình hóa bài toán tìm kiếm trong AI.
- Vận dụng BFS, UCS, Greedy và A*.
- Phân tích và đánh giá heuristic.
- Phân biệt heuristic admissible và consistent.
- So sánh tính đầy đủ, tối ưu và hiệu quả của các thuật toán tìm kiếm.

Câu 1. Tìm kiếm và heuristic

Một tác tử dạng xe cần thoát khỏi mê cung 5×5 (hình trong đề gốc: xuất phát ở (0,0) hướng Bắc, v = 0; đích ở (0,4) hướng Tây, v = 0). Tác tử luôn quay về một trong bốn hướng {Bắc, Nam, Đông, Tây}. Mỗi bước, tác tử có thể tiến về phía trước với vận tốc điều chỉnh được hoặc thực hiện thao tác rẽ.

Hành động di chuyển: Tăng tốc (v' = v + 1), Giảm tốc (v' = v − 1), Giữ tốc độ (v' = v); sau hành động xe đi v' ô. Hành động rẽ trái/phải đổi hướng 90°, chỉ được rẽ khi v = 0 và sau khi rẽ v vẫn bằng 0. Hành động không hợp lệ: va tường, v < 0 hoặc v > Vmax, giữ tốc độ khi v = 0, rẽ khi v ≠ 0. Chi phí đường đi = số hành động.

a) Chuỗi hành động ngắn nhất từ (0,0), hướng Bắc, v = 0 đến (0,4), hướng Tây, đi theo nhánh trái (qua ô (1,2)):

1. **Rẽ phải — (0,0), hướng Đông, v = 0**
2. **Tăng tốc — (1,0), v = 1**
3. **Giảm tốc — (1,0), v = 0**
4. **Rẽ trái — (1,0), hướng Bắc, v = 0**
5. **Tăng tốc — (1,1), v = 1**
6. **Giữ tốc độ — (1,2), v = 1**
7. **Giữ tốc độ — (1,3), v = 1**
8. **Giảm tốc — (1,3), v = 0**
9. **Rẽ trái — (1,3), hướng Tây, v = 0**
10. **Tăng tốc — (0,3), v = 1**
11. **Giảm tốc — (0,3), v = 0**
12. **Rẽ phải — (0,3), hướng Bắc, v = 0**
13. **Tăng tốc — (0,4), v = 1**
14. **Giảm tốc — (0,4), v = 0**
15. **Rẽ trái — (0,4), hướng Tây, v = 0 (ĐÍCH)**

Tổng chi phí = 15 hành động.

(Lý do: các cột (0,1),(0,2) bị tường bao kín nên phải đi qua hành lang x = 1; vì chỉ được rẽ khi v = 0 nên mỗi lần đổi hướng đều phải dừng; đoạn thẳng 3 ô cần ít nhất 4 hành động v' = 1,1,1,0.)

b) Kích thước không gian trạng thái (x, y, hướng, vận tốc):

M × N × 4 × (Vmax + 1)

c) Có thể tồn tại trạng thái không có nút con không?

☒ **Có** ☐ Không

Ví dụ: trạng thái (0,0), hướng Tây, v = 2 (giả sử Vmax ≥ 2; biên lưới coi như tường). Tăng tốc → đi 3 ô, va tường; Giữ tốc độ → đi 2 ô, va tường; Giảm tốc → đi 1 ô, va tường; không được rẽ vì v ≠ 0. Vậy không còn hành động hợp lệ nào.

d) Hệ số phân nhánh lớn nhất:

Hệ số phân nhánh lớn nhất = 3.

Ví dụ 1: (1,0), hướng Bắc, v = 0 → các hành động: Tăng tốc, Rẽ trái, Rẽ phải (Giữ tốc độ và Giảm tốc không hợp lệ khi v = 0).

Ví dụ 2: (1,1), hướng Bắc, v = 1 (0 < v < Vmax) → các hành động: Tăng tốc, Giữ tốc độ, Giảm tốc (không được rẽ vì v ≠ 0).

e) Khoảng cách Manhattan có là heuristic chấp nhận được không?

☐ Có ☒ **Không**

Vì mỗi hành động có thể đi nhiều hơn 1 ô (v' ≥ 2), chi phí thực có thể nhỏ hơn số ô cần đi. Phản ví dụ (lưới rộng/không tường, Vmax ≥ 2): xe ở (0,0), hướng Đông, v = 0, ô thoát ở (6,0) hướng Đông. h = Manhattan = 6, nhưng chi phí tối ưu = 5: Tăng tốc (đi 1 ô), Tăng tốc (2 ô), Giữ tốc độ (2 ô), Giảm tốc (1 ô), Giảm tốc (v = 0, dừng ở ô thứ 6) → 5 hành động < 6 nên h > h*, không admissible.

f) h = khoảng cách Manhattan / Vmax có admissible không?

☒ **Có** ☐ Không

Giải thích: mỗi hành động di chuyển tối đa Vmax ô, nên để đi quãng đường ít nhất bằng khoảng cách Manhattan cần ít nhất Manhattan / Vmax hành động (chưa tính các hành động rẽ, tăng/giảm tốc làm chi phí thật chỉ lớn hơn). Do đó h ≤ h*(s) với mọi s, và h = 0 ở đích → admissible.

g) A* Tree Search với heuristic không admissible: tính đầy đủ có thể thay đổi không?

☐ Có ☒ **Không**

Giải thích: với đồ thị hữu hạn, h không âm (hữu hạn) và chi phí bước dương, f = g + h tăng không giới hạn dọc đường vô hạn nên nút đích cuối cùng vẫn được mở rộng → A* vẫn đầy đủ.

h) A* Tree Search với heuristic không admissible: tính tối ưu có thể thay đổi không?

☒ **Có** ☐ Không

Giải thích: nếu h đánh giá quá cao (h > h*), một đường đi tối ưu có thể bị xếp f lớn và bị bỏ qua; A* có thể trả về đích qua đường chi phí cao hơn → mất tính tối ưu (ví dụ ở Câu 3: A* với h1 trả về đường chi phí 12 thay vì 11).

i) Lý do hợp lý khi dùng heuristic không admissible (chọn tất cả phương án phù hợp):

☒ **Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.**

☒ **Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.**

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

☒ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) với hệ số phân nhánh tối đa b. Mỗi nút n có chi phí tích lũy g(n), heuristic admissible h(n), độ sâu d(n). n_c là nút đích có chi phí nhỏ nhất, n_s là nút đích nông nhất. Viết bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc (không xử lý hòa).

a) Breadth-First Search (BFS).

Bất đẳng thức: d(n) < d(n_s)

b) Uniform-Cost Search (UCS).

Bất đẳng thức: g(n) < g(n_c)

c) A* Search, h là heuristic nhất quán.

Bất đẳng thức: g(n) + h(n) < g(n_c)

Câu 3. Tìm kiếm trên đồ thị

Đồ thị vô hướng (hình trong đề): A–B 3, A–C 4, B–C 2, B–D 3, B–E 5, C–E 2, D–E 1, E–F 3, E–G 6, F–G 2. A là bắt đầu, G là đích.

Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

a) Các đường đi mà thuật toán có thể trả về:

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS		**X**		
Greedy với h1				**A-B-E-G**
Greedy với h2	**X**			
A* với h1			**X**	

A* với h2		**X**		

Ghi chú: Greedy h1: A→B (6) → D/E (cùng 3, hòa) → E → G, đường A-B-E-G (hoặc A-B-D-E-G tùy cách phá hòa), cả hai đều không nằm trong ba đường đã liệt kê.

b) Chi phí đường đi tối ưu từ A đến G theo UCS:

Đáp án: **11 (đường A-C-E-F-G = 4 + 2 + 3 + 2)**

c) Heuristic h1:

Admissible: ☐ Có ☒ **Không** Consistent: ☐ Có ☒ **Không**

(h1(A) = 12 > h*(A) = 11 và h1(C) = 9 > h*(C) = 7 nên không admissible; do đó cũng không consistent.)

d) Heuristic h2:

Admissible: ☒ **Có** ☐ Không Consistent: ☐ Có ☒ **Không**

(h2 ≤ h* ở mọi nút: A 11/11, B 7/9, C 6/7, D 4/6, E 5/5, F 1/2, G 0/0 → admissible. Nhưng h2(A) = 11 > c(A,B) + h2(B) = 3 + 7 = 10 nên không consistent.)

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đúng/Sai (mọi hành động có chi phí ≥ ε > 0, mọi heuristic đều consistent):

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.
☐ Đúng ☒ **Sai** **(DFS tree search có thể lặp vô hạn trên chu trình.)**

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.
☒ **Đúng** ☐ Sai

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.
☒ **Đúng** ☐ Sai

Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.
☐ Đúng ☒ **Sai** **(Graph Search bỏ trạng thái trùng nên frontier không bao giờ lớn hơn.)**

Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.
☐ Đúng ☒ **Sai** **(Graph Search phải lưu tập explored nên bộ nhớ không còn như DFS.)**

Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.
☒ **Đúng** ☐ Sai

b) Điền heuristic admissible nhưng không consistent (S bắt đầu, G đích; S–A 1, S–C 2, A–G 3, C–G 2):

Trạng thái	h(s)
S	**4**
A	**2**
C	**2**
G	0

Kiểm tra: chi phí thật h*(S) = 4, h*(A) = 3, h*(C) = 2, nên 4 ≤ 4, 2 ≤ 3, 2 ≤ 2 → admissible. Nhưng h(S) = 4 > c(S,A) + h(A) = 1 + 2 = 3 → không consistent.